

CSCE 658 RANDOMIZED ALGORITHMS · MODULE 04 · LECTURE NOTES

Randomised Data Structures

*Expected balance instead of enforced balance.***MODULE OUTCOMES**

- Explain skip lists and derive their expected height.
- Explain treaps and their relationship to quicksort.
- Explain universal hashing and what it guarantees.
- Explain perfect and cuckoo hashing.
- Compare randomised against deterministic balancing honestly.

1 Skip lists and treaps

A sorted linked list, plus express lanes above it.
 Each element appears at level i with probability $1/2^i$ --
 flip a coin on insertion until it comes up tails.

SEARCH: start at the top level, move right while the next
 key is smaller than the target, drop down a level when it
 is not. Repeat until the bottom.

EXPECTED HEIGHT: $O(\log n)$. The probability that any element
 reaches level $c \log n$ is $n \cdot 2^{-(c \log n)} = n^{(1-c)}$, which
 is small for $c > 1$.

-- Module 02's per-event-plus-union-bound pattern, for
 the third time.

EXPECTED SEARCH TIME: $O(\log n)$. At each level you move
 right an expected constant number of steps before dropping,
 and there are $O(\log n)$ levels.

WHY IT IS ACTUALLY USED: there is NO REBALANCING CODE. No
 rotations, no colour invariants, no case analysis on
 uncle nodes. Insertion is: find the position, flip coins,
 splice in. Twenty lines against a red-black tree's two
 hundred.

AND IT IS FAR EASIER TO MAKE CONCURRENT, because there is
 no global rebalancing step that must be locked -- which is
 why concurrent skip lists appear in standard libraries and
 concurrent balanced trees are a research topic
 (CSCE 611, CSCE 678).

TREAPS: A BINARY SEARCH TREE WHOSE SHAPE IS A RANDOM QUICKSORT

Each node carries a key and a randomly assigned priority. The structure is a binary search tree with respect to the keys and a heap with respect to the priorities — and those two constraints together determine the shape uniquely, given the priorities.

And that shape is exactly the recursion tree of quicksort with random pivots: the highest-priority element is the root, which is the element quicksort would have chosen as its first pivot, and the recursion proceeds identically.

So the expected depth is $O(\log n)$ by precisely the same analysis as randomised quicksort (CSCE 629), and the analysis transfers without any modification — you do not need a new argument.

Which is a satisfying unification: skip lists, treaps, and randomised quicksort are three presentations of one analysis, and recognising that is worth considerably more than learning the three separately — it is the same level-by-level or depth-by-depth argument in three costumes.

2 Hashing

UNIVERSAL HASHING: A FAMILY WHERE ANY TWO KEYS COLLIDE WITH PROBABILITY $1/m$

A family H of hash functions into m buckets is *universal* if, for any two distinct keys, a uniformly chosen h from H collides on them with probability at most $1/m$.

That is all you need for the expected-chain-length analysis. The full independence of a truly random function is not required — the analysis only ever considers pairs — which is what makes a small, efficiently computable, explicitly storable family sufficient.

And the family can be tiny: $h(x) = ((ax + b) \bmod p) \bmod m$ for a prime p larger than the key range is universal, with a and b being the only randomness — two numbers, rather than a table of n random values.

Which is Module 12's limited-independence idea arriving early — you need pairwise independence rather than full independence, and pairwise independence is cheap to obtain and cheap to store. That observation is the whole reason hashing works in practice, since a truly random function cannot be stored.

Scheme	What it guarantees	What it costs
Universal hashing with chaining	$O(1)$ <i>expected</i> lookup.	Worst-case chain of $\log n / \log \log n$ (Module 03 §1), so the tail is not constant.
Perfect hashing (FKS)	$O(1)$ <i>worst-case</i> lookup.	Static key set only, and the construction is randomised (Las Vegas — retry until it succeeds).
Cuckoo hashing	$O(1)$ worst-case lookup, and the set may be dynamic.	Insertion may cascade and may require a full rehash; the load factor must stay below about 0.5 for two choices (§3).

Scheme	What it guarantees	What it costs
Linear probing	Excellent cache behaviour — the fastest in practice for many workloads.	The analysis needs 5-wise independence, which was only established relatively recently and is why its theoretical status lagged its practical use.
Bloom filter	$O(1)$ membership test with one-sided error.	No deletion; false positives only and never false negatives — which is what makes it usable as a pre-filter (Module 01 §2).

Perfect and cuckoo hashing give worst-case $O(1)$ lookup, which chaining does not — and both achieve it by using the randomness in the *construction* rather than in the *query*. That construction-versus-query distinction is the useful insight here, and it generalises: when reads vastly outnumber writes, moving the probabilistic work to build time converts an expected guarantee into a worst-case one.

3 Cuckoo hashing

CUCKOO HASHING GIVES WORST-CASE CONSTANT LOOKUP BY MOVING THE WORK TO INSERTION

Each key has exactly two possible positions, given by two hash functions. A lookup checks both. So lookup is worst-case two probes, always — no chains, no probing sequences, no tail.

Insertion places the key in one of its two positions, evicting whatever occupant was there; the evicted key is then placed in its own alternative position, evicting in turn — possibly cascading for some distance, hence the name.

The cascade terminates with high probability as long as the load factor stays below about 0.5 for two hash functions (higher for more functions or for buckets holding several items), and above that threshold the structure must be rebuilt with fresh hash functions.

So the randomness has moved from the query to the construction — which is the right place for it when reads vastly outnumber writes, and is the same reasoning as perfect hashing. And the two-position structure is Module 03's power of two choices, applied to hashing: the insertion is choosing the better of two locations, and the exponential improvement in the load tail is what makes the scheme work.

4 The honest comparison

	Randomised (skip list, treap)	Deterministic (red-black, B-tree)
Guarantee	Expected, with a tail bound.	Worst case.
Code size	Much smaller — no rebalancing logic at all.	Large case analysis, and a frequent source of bugs.
Concurrency	Easier — no global rebalancing step to serialise.	Hard, and lock-free variants are genuinely research-level.
Cache and disk behaviour	Worse — pointer chasing with poor locality.	B-trees are far better, because a node is sized to a cache line or a disk block.
Adversarial safety	Safe provided the randomness is hidden from the adversary (Module 01 §1).	Safe always, since there is nothing to learn.
What ships	Skip lists in concurrent ordered maps.	B-trees in every database and filesystem.

The cache row is why B-trees dominate storage and skip lists do not — the asymptotics are identical and the constants are not, which is exactly CSCE 637 Module 01 §4's point about what asymptotic analysis hides. Being honest that the deterministic structure frequently wins is important, because a course on randomised algorithms has an obvious bias to resist.

WHICH TO CHOOSE

A concurrent in-memory ordered map: skip list. The absence of global rebalancing is decisive, and it is why concurrent skip lists appear in standard libraries while concurrent balanced trees generally do not.

An on-disk or cache-sensitive ordered structure: B-tree. Node-per-block locality beats everything else, and the asymptotics do not distinguish the candidates at all.

A static set needing worst-case lookup: perfect hashing. Dynamic with the same requirement: cuckoo hashing (§3).

And for a sketch, a prototype, or a teaching implementation: the randomised one, always — twenty lines that you can read and verify beats two hundred that you cannot, and the expected guarantee is almost certainly adequate for a prototype. **Which is Module 01 §1's second use of randomness, and it is the one most often undervalued.**

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Pugh — Skip Lists: A Probabilistic Alternative to Balanced Trees (free)	§1's structure, in the original — short, and the argument for the code-size benefit is made explicitly.
Seidel & Aragon — Randomized Search Trees (free)	§1's treaps , including the quicksort correspondence.
Carter & Wegman — Universal Classes of Hash Functions (free)	§2's definition , and the construction that made hashing analysable.
Pagh & Rodler — Cuckoo Hashing (free)	§3's scheme , with the load-factor threshold derived.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Implement a skip list** and measure its height distribution against the predicted $O(\log n)$.
- 2 **Count the lines** against a red-black tree implementation.
- 3 **Implement a treap** and verify its shape matches a randomised quicksort recursion tree on the same input.
- 4 **Implement universal hashing** with the $((ax+b) \bmod p) \bmod m$ family and measure collision rates.
- 5 **Verify the $1/m$ pairwise collision bound** empirically.
- 6 **Implement perfect hashing** for a static set and measure the construction retries.

- 7 **Implement cuckoo hashing** and plot the insertion cascade length against the load factor.
- 8 **Find the threshold empirically** and compare against 0.5.
- 9 **Benchmark a skip list against a B-tree** on a large dataset and report the cache-miss rates.
- 10 **Make a skip list concurrent** and then attempt the same for a red-black tree. Report the difference in effort.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Describe a skip list and derive its expected height.
- 2 What is its actual benefit, and what follows for concurrency?
- 3 What determines a treap's shape, and what is it equal to?
- 4 Define universal hashing and say what it does not require.
- 5 Why can the family be tiny?
- 6 Compare five hashing schemes and their guarantees.
- 7 What distinction do perfect and cuckoo hashing share?
- 8 Explain cuckoo hashing and its load threshold.
- 9 Compare randomised against deterministic balancing on six axes.
- 10 Why do B-trees dominate storage?